Compte rendu du TP n°3

Analyse du courant d'un servomoteur : détection de défauts d'engrenages et estimation du RUL

Mariem Ben Attia

1. Objectifs et dispositif

Ce TP exploite le courant d'un servomoteur Futaba S3003 (alimenté à 5 V, réduction 278:1, environ 50 tr/min en sortie) pour détecter des défauts dans le train d'engrenages par analyse de signature du courant moteur (MCSA). Les objectifs sont :

- calculer la vitesse nominale, les fréquences d'intérêt et les bandes de défaut, puis extraire des métriques spectrales ;
- construire un nuage de points permettant de classer les données saines et défectueuses ;
- envoyer les caractéristiques vers un canal ThingSpeak et estimer en continu la durée de vie résiduelle (RUL) avec MATLAB.

Les données sont dans le fichier mcsaData.mat : 10 acquisitions saines et 10 acquisitions défectueuses (défauts sur les engrenages étagés G2 et G3). Le courant est échantillonné à 1,5 kHz par un Arduino Uno, avec le signal du tachymètre (8 impulsions par tour de sortie).

2. Partie 1 : analyse MCSA du courant moteur

2.1 Structure des données (Q1)

La structure servoData contient deux cell arrays 10×1, healthyData et faultyData. Chaque cellule est un timetable de 16384 lignes et 2 colonnes, plus la colonne de temps :

- **Time** : instant de l'échantillon (durée) ;
- **Pulse** : signal du tachymètre (0 ou 1), 8 impulsions par tour de sortie ;
- **Signal** : courant du moteur, en mA.

Le pas de temps est dt = $6{,}6667 \cdot 10^{-4}$ s, soit une fréquence d'échantillonnage fs = 1/dt = 1500 Hz (valeur confirmée par le script : « Fs déduite = 1500.0 Hz »). Chaque acquisition dure donc 16384/1500 ≈ 10,9 s.

```
Fs déduite = 1500.0 Hz
rpmHealthy = 49.85 rpm (théorique : 50 rpm)
rpmFaulty  = 49.53 rpm (théorique : 50 rpm)

FS =

  231.0207   37.2614    7.4523    2.1292    0.8309
```

Figure 1 – Fs déduite, vitesses moyennes et vecteur FS

2.2 Visualisation du courant et du tachymètre (Q2)

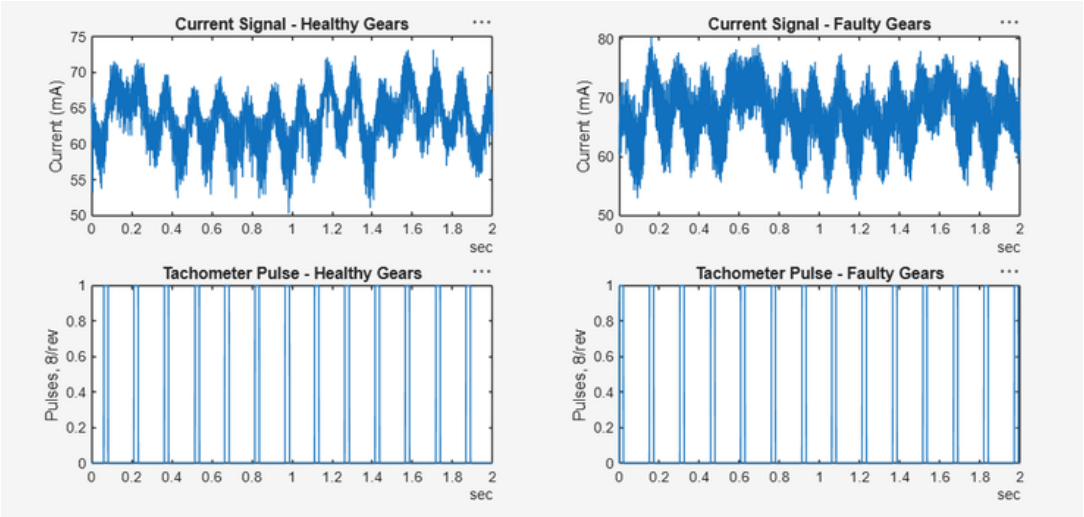


Figure 2 – Courant et impulsions du tachymètre (sain / défectueux), 0 à 2 s

Le courant oscille autour de 60 à 65 mA dans les deux cas. Le signal défectueux a une enveloppe un peu plus ample (jusqu'à 80 mA contre 75 mA pour le sain). Les impulsions du tachymètre sont régulières dans les deux cas : environ 6,6 impulsions par seconde, soit 8 impulsions par tour à environ 50 tr/min.

2.3 Vitesses nominales (Q3 et Q4)

La fonction tachorpm, appelée avec fs = 1500 Hz et 8 impulsions par tour, fournit la vitesse instantanée à partir du signal du tachymètre.

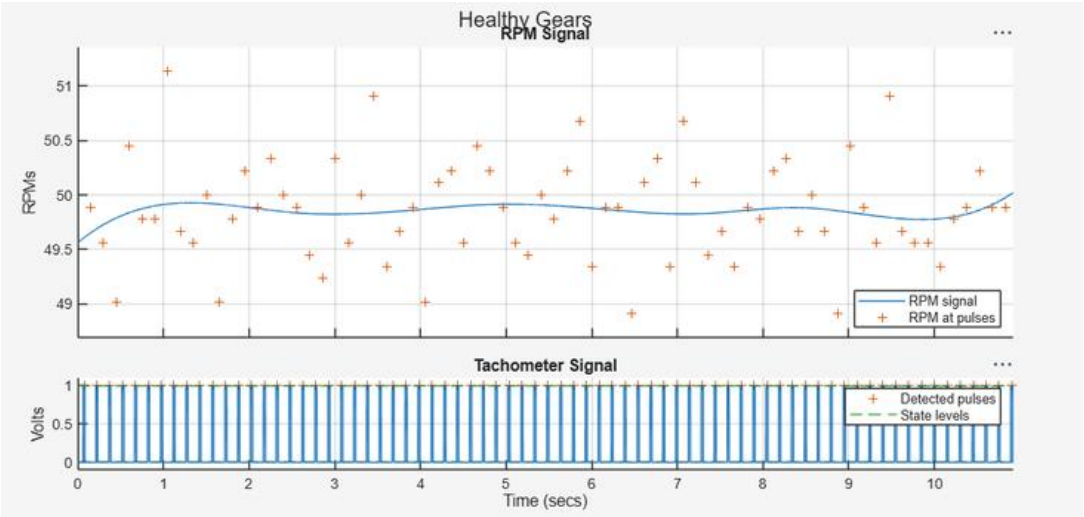


Figure 3 – Vitesse et tachymètre, engrenages sains

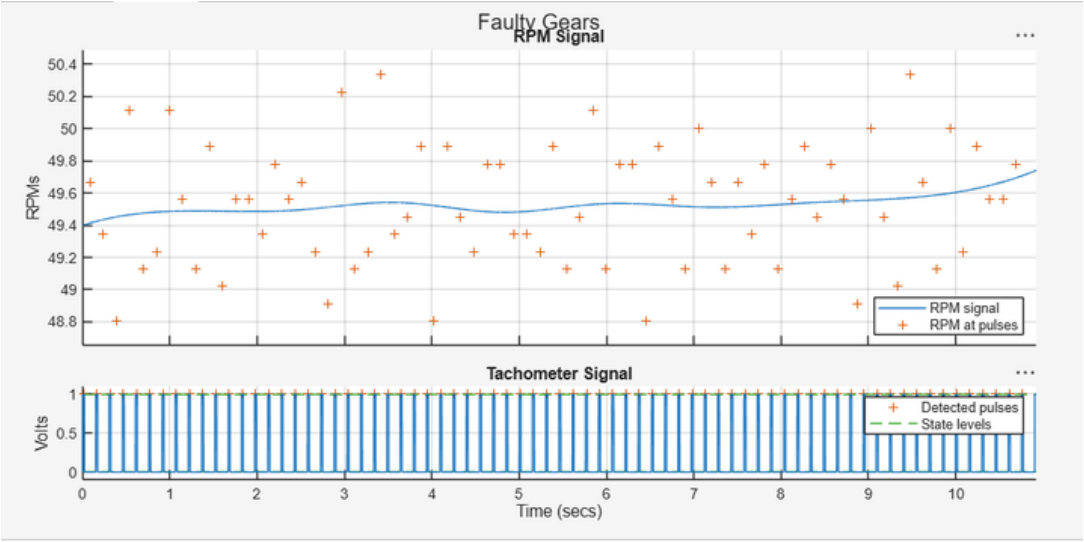


Figure 4 – Vitesse et tachymètre, engrenages défectueux

Jeu de données	Vitesse moyenne (tr/min)	Théorique (tr/min)	Écart relatif
Sain (rpmHealthy)	49,85	50	−0,3 %
Défectueux (rpmFaulty)	49,53	50	−0,9 %

Les deux vitesses sont très proches de la valeur théorique de 50 tr/min. Le jeu défectueux tourne un peu plus lentement que le sain (écart d'environ 0,3 tr/min, soit 0,64 %), ce qui traduit un frottement ou une charge supplémentaire liés aux défauts. Cet écart est faible mais mesurable, et il se retrouve sur le spectre (voir 2.5).

2.4 Fréquences d'intérêt et bandes de défaut (Q5 et Q6)

À partir de rpmHealthy et des nombres de dents (G1 = 62, G2 = 50, G3 = 35, G4 = 41 ; P1 = P2 = P3 = 10, P4 = 16), on remonte du dernier arbre vers le moteur :

Fréquence	Valeur calculée (Hz)	Tableau 1 du TP (Hz)	Arbre / engrènement
FS1	231,02	231,69	moteur (engrènement P1–G1)
FS2	37,26	37,37	engrenage G1/P2
FS3	7,45	7,47	engrenage G2/P3
FS4	2,13	2,14	engrenage G3/P4
FS5	0,83	0,83	arbre de sortie

Les valeurs calculées sont cohérentes avec le tableau 1, ce qui valide la mesure de vitesse. La fonction faultBands construit ensuite les bandes autour de FS1 (avec ses deux premières harmoniques et les bandes latérales 0:1 de FS2), de FS2 (avec bandes latérales de FS3), de FS3 et de FS4 (avec leurs deux premières harmoniques). On obtient FB1 (6×2), FB2 (6×2), FB3 (2×2) et FB4 (2×2), soit 16 bandes au total (matrice FB de 16×2).

2.5 Spectres de puissance (Q7 et Q8)

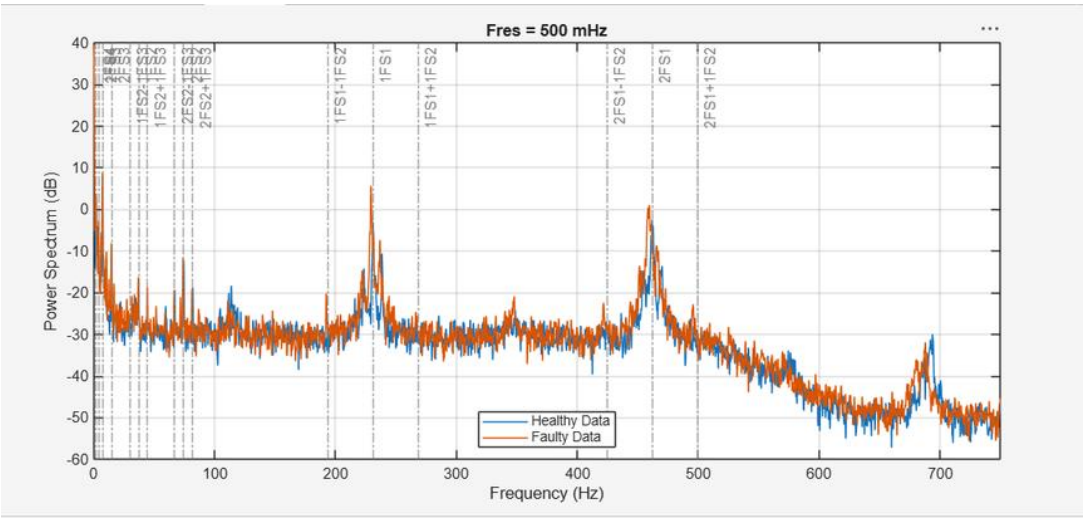


Figure 5 – Spectres de puissance du courant (bleu : sain, orange : défectueux) avec les bandes de défaut

Observations :

- Les deux spectres ont les mêmes raies principales : FS1 (≈ 231 Hz) et 2FS1 (≈ 462 Hz), ainsi qu'un plancher de bruit voisin de −30 dB. Le pic vers 690 Hz est présent dans les deux cas et se situe hors des bandes de défaut.
- Autour de FS1, le pic du signal défectueux est nettement plus haut (environ +5,5 dB contre −3 dB pour le sain), plus large, et accompagné de bandes latérales plus marquées.
- En basses fréquences (0 à 100 Hz), le signal défectueux présente des pics plus nets autour de FS2, FS3 et FS4 et de leurs harmoniques, qui correspondent aux engrenages G2 et G3 où les défauts ont été introduits.

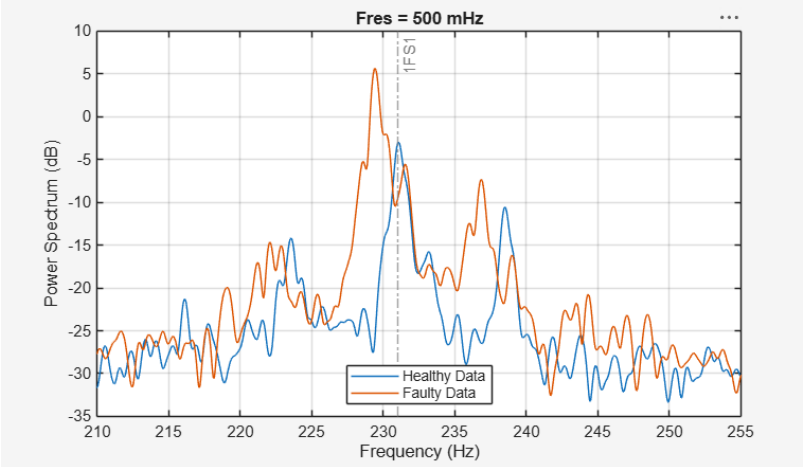


Figure 6 – Zoom autour de FS1 (210 à 255 Hz)

Sur le zoom, le pic du signal défectueux est décalé vers la gauche (≈ 229,5 Hz) par rapport à la ligne FS1 (231 Hz) qui est calculée avec rpmHealthy. Ce décalage est confirmé par le calcul FS1 = (62/10)·(50/10)·(35/10)·(41/16)·rpmFaulty/60 = 229,50 Hz. Il vient de la vitesse un peu plus faible du jeu défectueux, et le pic reste à l'intérieur de la bande de défaut.

Conclusion : le courant du moteur contient bien la signature des défauts des engrenages G2 et G3 : amplitude accrue et élargissement de la raie FS1, bandes latérales plus marquées, pics plus nets aux fréquences des arbres concernés. La MCSA permet donc de détecter ces défauts sans capteur de vibration. Les différences restent toutefois modestes à l'œil, ce qui justifie le calcul de métriques spectrales.

2.6 Métriques spectrales (Q9 à Q11)

Les PSD et les grilles de fréquences (psdHealthy, fHealthy, psFaulty, fFaulty, 12001 points chacune) sont calculées avec pspectrum (résolution 0,5 Hz). La fonction faultBandMetrics, appliquée avec les 16 bandes, renvoie une table d'une ligne par signal et de 49 colonnes : pour chacune des 16 bandes, PeakAmplitude, PeakFrequency et BandPower (16×3 = 48), plus TotalBandPower, somme des puissances de bandes.

Métrique	Sain	Défectueux	Rapport (déf./sain)
PeakAmplitude2 (FS1)	0,508	3,692	× 7,3
BandPower2 (FS1)	0,467	2,998	× 6,4
PeakAmplitude5 (2FS1)	0,550	1,297	× 2,4
BandPower5 (2FS1)	0,895	3,219	× 3,6
PeakAmplitude13 (FS3)	5,790	7,793	× 1,3
PeakAmplitude15 (FS4)	0,790	2,421	× 3,1
BandPower15 (FS4)	0,144	0,441	× 3,1
TotalBandPower	3,974	9,984	× 2,5

Les métriques les plus discriminantes sont PeakAmplitude2 et BandPower2 (FS1), puis PeakAmplitude5 et BandPower5 (2FS1), PeakAmplitude15 et BandPower15 (FS4) et la puissance totale TotalBandPower. À l'inverse, les bandes latérales de FS1 (métriques 1, 3, 4, 6) et les bandes de FS2 et FS3 (métriques 7 à 12) ont des valeurs du même ordre pour les deux signaux. Les fréquences des pics sont presque identiques, sauf pour FS1 (231,06 → 229,44 Hz) et 2FS1 (462 → 459,69 Hz), ce qui confirme le décalage de vitesse. Ces rapports sont calculés sur un seul couple de signaux : la séparation sur l'ensemble des 20 jeux est examinée ci-dessous.

2.7 Classification sain / défectueux (Q12)

Les 49 métriques ont été calculées pour les 20 acquisitions (table allMetrics de 20×50, avec la colonne de condition). Le nuage de points ci-dessous représente PeakAmplitude2 en fonction de TotalBandPower.

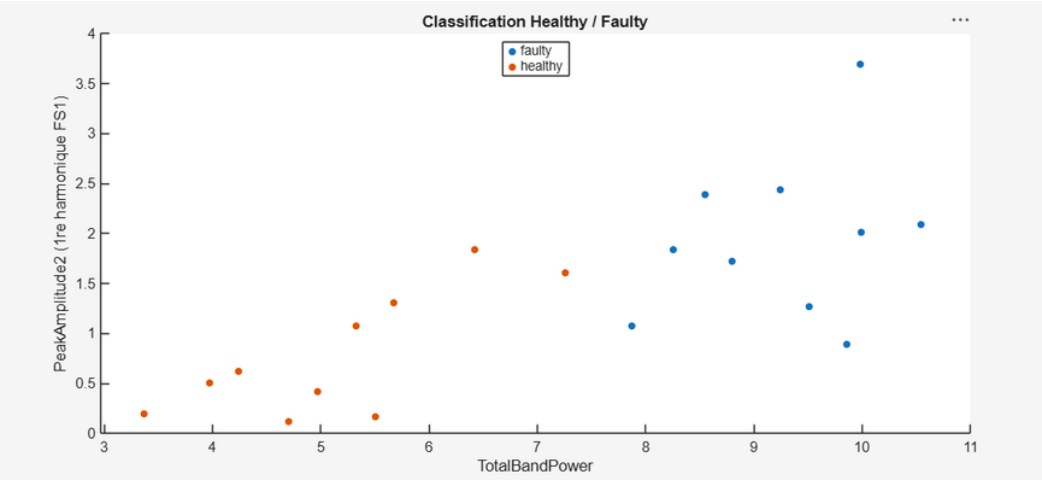


Figure 7 – Nuage de points : TotalBandPower et PeakAmplitude2 (healthy / faulty)

La puissance totale sépare bien les deux classes : les jeux sains ont TotalBandPower entre environ 3,4 et 7,3, les jeux défectueux entre environ 7,9 et 10,5. Un seuil voisin de 7,5 à 7,6 permet de les distinguer sur ces 20 acquisitions. PeakAmplitude2 seule recouvre les deux classes (de 0,1 à 1,8 pour le sain, de 0,9 à 3,7 pour le défectueux) : elle apporte une information utile en complément, mais ne suffit pas à elle seule. Cette

séparation peut être exploitée par un classifieur (outil Classification Learner de MATLAB) avec ces deux variables ou davantage.

3. Partie 2 : estimation du RUL en temps réel

3.1 Canal ThingSpeak (Q1 à Q3)

Le canal test_servo_Motor_currect_features (ID 3518312) a été créé avec six champs : Field 1 PeakAmplitude2, Field 2 PeakAmplitude5, Field 3 TotalBandPower, Field 4 courant moyen, Field 5 vitesse moyenne (tr/min) et Field 6 RUL. Les clés API d'écriture et de lecture ont été relevées.

3.2 Envoi périodique des caractéristiques (Q4 et Q5)

Un timer MATLAB (fixedSpacing, période 60 s) appelle la fonction sendSyntheticFeaturesToThingSpeak. À chaque appel, la fonction choisit au hasard un jeu sain parmi les dix, calcule la vitesse moyenne (tachorpm), les fréquences FS1 à FS4 et les bandes de défaut, la PSD (pspectrum), les 49 métriques (faultBandMetrics), puis envoie le vecteur de caractéristiques avec thingSpeakWrite. Exemple d'exécution :

```
Dataset choisi : healthyData{10,1}
Fs = 1500.00 Hz
Average RPM = 49.886 rpm

Frequencies:
FS1 = 231.1643 Hz
FS2 = 37.2846 Hz
FS3 = 7.4569 Hz
FS4 = 2.1305 Hz
FS5 = 0.8314 Hz

Features envoyées :
PeakAmplitude2       = 1.837254
PeakAmplitude5       = 1.080739
TotalBandPower       = 6.426596
Average Current      = 61.920472 mA
Average Speed        = 49.885969 rpm

Features envoyées à ThingSpeak.
```

Figure 8 – Sortie MATLAB lors d'un envoi (jeu healthyData{10,1})

La vitesse (49,89 tr/min) et les fréquences (FS1 = 231,16 Hz) sont cohérentes avec la partie 1. Les caractéristiques envoyées sont : PeakAmplitude2 = 1,837, PeakAmplitude5 = 1,081, TotalBandPower = 6,427, courant moyen = 61,92 mA et vitesse moyenne = 49,886 tr/min.

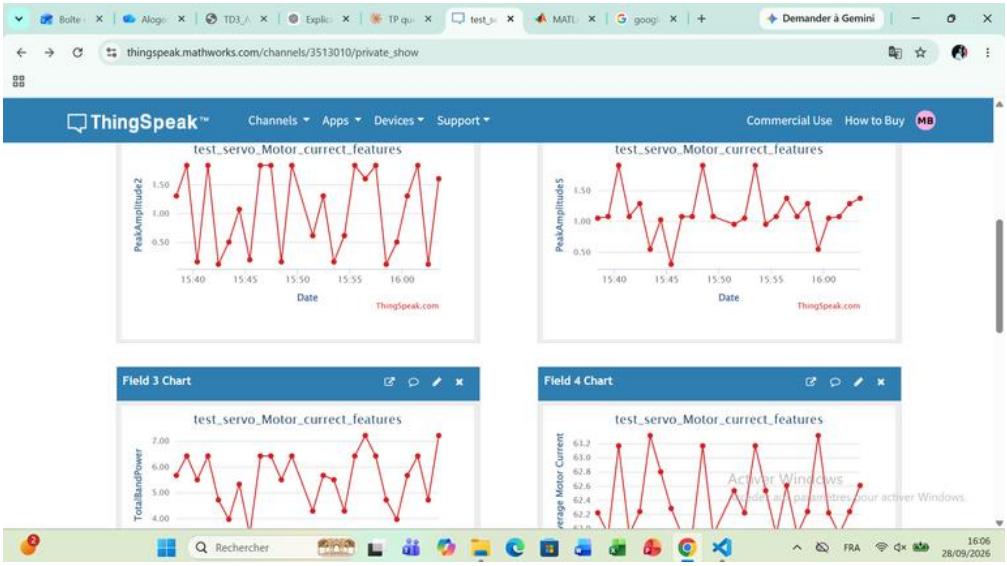


Figure 9 – Tableau de bord ThingSpeak : champs 1 à 4 au cours du temps

Les courbes fluctuent d'un point à l'autre, ce qui est normal puisque chaque point provient d'un jeu sain tiré au hasard. Les valeurs restent dans la plage des jeux sains (TotalBandPower entre 4 et 7,2).

3.3 Modèle de dégradation exponentielle (Q6 et Q7)

La fonction exponentialDegradationModel modélise l'indice de santé par $h(t) = \phi + \theta \cdot \exp(\beta \cdot t + \varepsilon)$, où θ et β sont des variables gaussiennes dont on fixe la distribution a priori (Theta, ThetaVariance, Beta, BetaVariance) et ε est un bruit de variance NoiseVariance. À chaque observation, les paramètres sont mis à jour par une approche bayésienne ; le RUL est le temps restant avant que l'indice de santé franchisse un seuil.

Ce choix est adapté au train d'engrenages : l'usure et la fatigue des engrenages en nylon s'accélèrent typiquement de façon exponentielle, et le modèle s'actualise en ligne avec peu de calculs, sans historique complet de pannes, ce qui convient au traitement en périphérie.

Indice de santé : grandeur scalaire, idéalement monotone, qui résume l'état du composant. Ici, c'est TotalBandPower (champ 3), et le seuil de défaillance du système est fixé à 10, ce qui correspond à la valeur observée sur les jeux défectueux (≈ 10).

Le modèle est créé avec Theta = 1, ThetaVariance = 100, Beta = 1, BetaVariance = 100, NoiseVariance = 0,01 et une unité de temps en heures. Un second timer (60 s) appelle readFeaturesFromThingSpeak.

3.4 Mise à jour du RUL (Q8)

La fonction readFeaturesFromThingSpeak utilise deux variables persistantes : timestamp (début de la fenêtre de lecture, initialisée à 10 jours en arrière puis avancée après chaque lecture) et hplot (handle du tracé, pour compléter la courbe sans effacer les valeurs précédentes). Elle lit les dernières données avec thingSpeakRead (format timetable), puis pour chaque ligne :

- `update(mdl, T(i,:))` met à jour les paramètres θ et β du modèle avec la nouvelle observation de l'indice de santé (apprentissage en ligne) ;
- `predictRUL(mdl, threshold)` extrapole la courbe de dégradation avec les paramètres courants et renvoie le temps restant avant d'atteindre le seuil (10), c'est-à-dire le RUL estimé.

La dernière valeur estimée est enfin écrite dans le champ 6 du canal (thingSpeakWrite avec hours(estRULAct)).

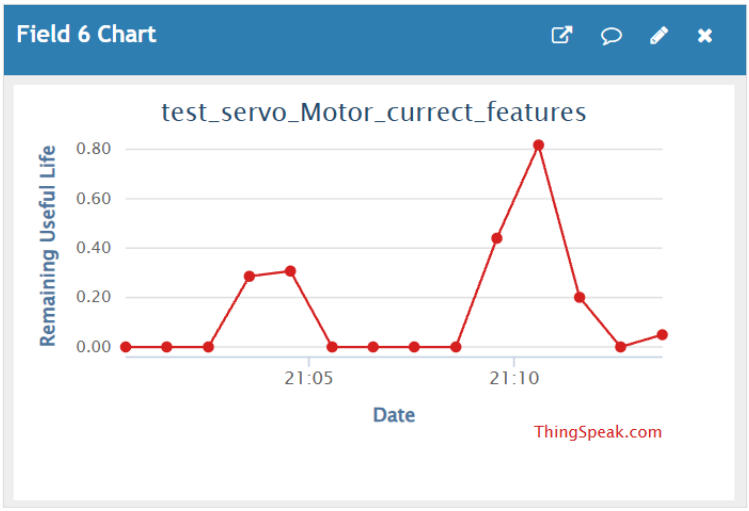


Figure 10 – Champ 6 : RUL estimé en temps réel

La courbe du champ 6 est irrégulière : des valeurs nulles alternent avec de petits pics (≈ 0,3 h puis ≈ 0,8 h). Ce comportement est attendu : les données injectées proviennent de jeux sains choisis au hasard, donc l'indice de santé ne montre aucune tendance de dégradation, et le modèle, mis à jour avec très peu de points et encore proche de ses valeurs a priori, ajuste surtout le bruit. Le RUL vaut 0 tant qu'aucune pente significative n'est détectée. Ce résultat valide la chaîne complète (acquisition, envoi, lecture, estimation, réécriture), mais pas la précision de la prédiction. Avec une réelle dégradation (indice croissant vers 10), on attend un RUL décroissant.

4. Tâches complémentaires

4.1 Jauge Google Gauge

Un plugin Google Gauge (RUL Gauge) a été ajouté au tableau de bord. Il lit la dernière valeur du champ 6 (requête fields/6/last.json), l'affiche en heures sur une échelle de 0 à 24 h, avec une zone rouge (RUL < 8 h), jaune (8 à 16 h) et verte (16 à 24 h), rafraîchie toutes les 15 s.

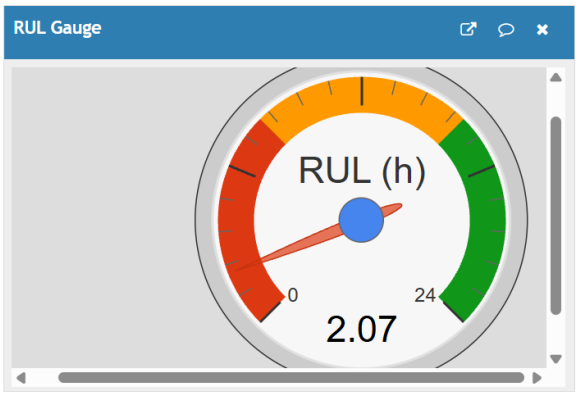


Figure 11 – Jauge du RUL (2,07 h, zone critique)

La jauge affiche 2,07 h : l'aiguille est dans la zone rouge, ce qui est cohérent avec les valeurs de RUL estimées.

4.2 Alerte RUL < 8 h

Une MATLAB Analysis nommée RUL_Alert lit la dernière valeur du champ 6. Si elle est inférieure à 8 h, elle écrit un message d'état dans le canal (« ALERT: servo gear train failure imminent, RUL = … h ») et envoie un

e-mail via l'API d'alertes de ThingSpeak. Elle est déclenchée toutes les 10 minutes par une application TimeControl.

Name:	RUL_check_10min
Frequency:	Every 10 minutes
Time Zone:	UTC (edit)
Last Ran:	
Run At:	2026-10-01 8:30 pm
Fuzzy Time:	± 0 minutes
MATLAB Analysis:	RUL_Alert
Created:	2026-10-01 8:29 pm

Figure 12 – Configuration du TimeControl RUL_check_10min

Le TimeControl RUL_check_10min est configuré avec une fréquence de 10 minutes, l'analyse RUL_Alert comme action, une heure de première exécution à 20:30 UTC et sans décalage aléatoire (fuzzy time = ± 0 min). Comme le RUL estimé (de 0 à environ 2 h) est inférieur à 8 h, la condition est vérifiée et l'alerte doit se déclencher à chaque exécution.

[À compléter : insérer ici la capture du message d'état dans le canal et celle de l'e-mail reçu (objet « Servo RUL alert »).]

5. Conclusion

Ce TP a montré que l'analyse du courant d'un servomoteur permet de détecter des défauts d'engrenages sans capteur de vibration. La vitesse mesurée (≈ 49,8 tr/min) a permis de construire les fréquences d'intérêt, conformes au tableau théorique, et les bandes de défaut. Les spectres et les métriques spectrales du signal défectueux sont nettement plus élevés autour de FS1 (amplitude × 7), 2FS1 et FS4, et la puissance totale des bandes (≈ 10 contre ≈ 4) sépare clairement les 20 acquisitions en deux classes.

La puissance totale a ensuite servi d'indice de santé pour une estimation du RUL en ligne avec un modèle de dégradation exponentielle, alimenté en temps réel via ThingSpeak, avec visualisation (courbe, jauge) et alerte (React/TimeControl et e-mail). Les données injectées étant saines, le RUL estimé reste instable et ne décroît pas : la chaîne fonctionne, mais une évaluation de la qualité de la prédiction demanderait des données de dégradation réelles (indice croissant progressivement vers le seuil).

Annexe : codes principaux

Fréquences d'intérêt et bandes de défaut

```
G4 = 41; G3 = 35; G2 = 50; G1 = 62; P4 = 16; P3 = 10; P2 = 10; P1 = 10;
FS5 = rpmHealthy/60;  FS4 = G4/P4*FS5;  FS3 = G3/P3*FS4;
FS2 = G2/P2*FS3;      FS1 = G1/P1*FS2;
[FB1,info1] = faultBands(FS1,1:2,FS2,0:1);
[FB2,info2] = faultBands(FS2,1:2,FS3,0:1);
[FB3,info3] = faultBands(FS3,1:2);
[FB4,info4] = faultBands(FS4,1:2);
FB = [FB1; FB2; FB3; FB4];
[psdHealthy,fHealthy] = pspectrum(healthyCurrent,fs,"FrequencyResolution",0.5);
metricsHealthy = faultBandMetrics(psdHealthy,fHealthy,FB);
```

Fonction d'envoi vers ThingSpeak

```
function sendSyntheticFeaturesToThingSpeak()
load("mcsaData.mat","servoData");  ind = randi(10,1,1);
TestData = servoData.healthyData{ind,1};
TestCurrent = TestData.Signal;  TestTacho = TestData.Pulse;  Fs = 1500;
rpm = mean(tachorpm(TestTacho,Fs,"PulsesPerRev",8),"omitnan");
% ... FS1..FS4, FB1..FB4 comme ci-dessus ...
[ps1,f1] = pspectrum(TestCurrent,Fs,"FrequencyResolution",0.5);
m = faultBandMetrics(ps1,f1,FB);
features = [m.PeakAmplitude2, m.PeakAmplitude5, m.TotalBandPower, mean(TestCurrent), rpm];
thingSpeakWrite(channelID,features,"Fields",1:numel(features),"WriteKey",writeKey);
end
```

Modèle de dégradation et estimation du RUL

```
mdl = exponentialDegradationModel("Theta",1,"ThetaVariance",100, ...
   "Beta",1,"BetaVariance",100,"NoiseVariance",0.01, ...
   "LifeTimeVariable",T.Properties.DimensionNames{1}, ...
   "DataVariables",T.Properties.VariableNames{3},"LifeTimeUnit","hours");
threshold = 10;
for i = 1:height(T)
   update(mdl,T(i,:));
   estRUL(i) = predictRUL(mdl,threshold);
end
thingSpeakWrite(channelID,hours(estRUL(end)),"Fields",6,"WriteKey",writeKey);
```

Analyse d'alerte (MATLAB Analysis, exécutée toutes les 10 min)

```
rul = thingSpeakRead(channelID,"Fields",6,"NumPoints",1,"ReadKey",readKey);
if ~isempty(rul) && ~isnan(rul) && rul < 8
   msg = sprintf("ALERT: servo gear train failure imminent, RUL = %.2f h",rul);
   thingSpeakWrite(channelID,rul,"Fields",6,"Status",msg,"WriteKey",writeKey);
   opts = weboptions("HeaderFields",["ThingSpeak-Alerts-API-Key",alertKey]);
   webwrite("https://api.thingspeak.com/alerts/send","subject","Servo RUL
alert","body",msg,opts);
end
```